

Bresenham Circle Drawing Theory (Easy Step-by-Step)

This note explains the theory behind circle drawing in:

- 03_bresenham_circle.py

The algorithm is also called the Midpoint Circle Algorithm.

1) What Problem Are We Solving?

We need to draw a circle of radius r centered at (x_c, y_c) on a pixel grid.

A perfect circle is continuous, but the screen is discrete.

So, at each step, we choose the best pixel to approximate the true circle.

2) Circle Equation and Decision Logic

For a circle centered at origin:

$$x^2 + y^2 = r^2$$

Define:

$$F(x, y) = x^2 + y^2 - r^2$$

1. $F < 0$ means point is inside the circle.
2. $F = 0$ means point is on the circle.
3. $F > 0$ means point is outside the circle.

At each step we test a midpoint to decide whether y should stay same or decrease.

3) Biggest Idea: 8-Way Symmetry

If one point (x, y) is on the circle, then these 8 points are also on the circle around center (x_c, y_c) :

1. $(x_c + x, y_c + y)$
2. $(x_c + x, y_c - y)$
3. $(x_c - x, y_c + y)$
4. $(x_c - x, y_c - y)$
5. $(x_c + y, y_c + x)$
6. $(x_c + y, y_c - x)$
7. $(x_c - y, y_c + x)$
8. $(x_c - y, y_c - x)$

So we compute one octant and mirror to all 8 parts.
This is the key speed-up.

4) Initial Values and Update Rules

Start from top point of circle in first octant:

- $x = 0$
- $y = r$
- $p = 3 - 2*r$

Loop while $x \leq y$:

1. Plot 8 symmetric points.
2. If $p < 0$:
 - choose East pixel (x increases, y same)
 - $p = p + 4*x + 6$
3. Else:
 - choose South-East pixel (x increases, y decreases)
 - $p = p + 4*(x - y) + 10$
 - $y = y - 1$
4. $x = x + 1$

Stop when $x > y$ (passed 45 degrees in first octant).

5) Step-by-Step Algorithm

Step 1: Input

Given center (x_c, y_c) and radius r

Step 2: Handle trivial case

If $r == 0$, circle is just one point: (x_c, y_c)

Step 3: Initialize

- $x = 0$
- $y = r$
- $p = 3 - 2*r$

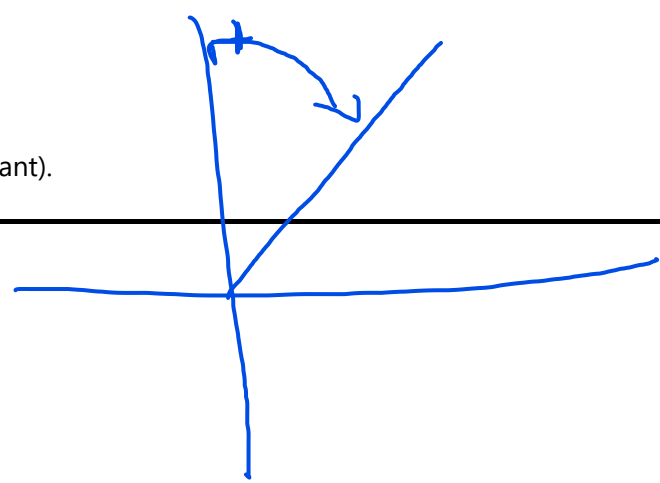
Step 4: Repeat until $x > y$

1. Plot symmetric points for (x, y)
2. Apply decision using p
3. Update p
4. Increment x

$x_1 = 0$
 $y_1 = r = 10$

$25 + p \Rightarrow d_0$

$d_0 = 3 - 2*r$
 d_0



$if \ d_k < 0$

$x_{k+1} = x_k + 1$
 $y_{k+1} = y_k$
 $d_{k+1} = d_k + 4*x_{k+1} + 6$

Step 5: Done

Collected points form the raster circle.

6) Worked Example ($r = 5$)

Initial:

- $x = 0$
- $y = 5$
- $p = 3 - 10 = -7$

Step	x	y	p before	Decision	p after
0	0	5	-7	start	-7
1	1	5	-7	$p < 0$, y same	-1
2	2	5	-1	$p < 0$, y same	9
3	3	4	9	$p >= 0$, y--	7
4	4	3	7	$p >= 0$, y--	13

Stop after this because next x would exceed y.

At every row above, plot all 8 symmetric points around (x_c, y_c) .

7) Why It Is Efficient

1. Integer math only.
2. One-octant computation with 8-way mirroring.
3. No trigonometric functions (no sin/cos required).
4. Very suitable for teaching and low-level rendering.

8) Tricky Scenarios

1. Small radius ($r = 1$ or 2):
 - many symmetric points overlap.
 - deduplicate points to avoid repeated plotting.
2. $x == y$ case:
 - points on diagonal can duplicate; deduplicate helps.
3. Radius from mouse input can be 0:
 - handle as validation or single-point circle.
4. Coordinate-system confusion:
 - Cartesian vs screen coordinates must be converted correctly.

$$d_k \geq 0$$

$$\begin{aligned} x_{k+1} &= x_k + 1 \\ y_{k+1} &= y_k \end{aligned}$$

$$d_{k+1} = d_k + 4x_{k+1} - y_{k+1} + 10$$

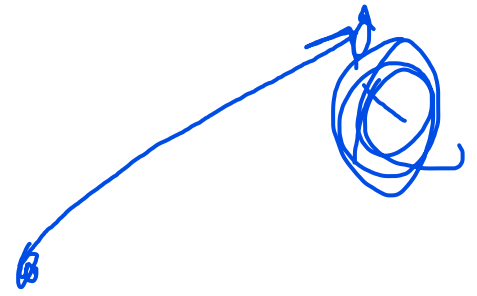
$$d_{k+1} = d_k + 4x_{k+1} - y_{k+1} + 10$$

$$\begin{aligned} 7, 7 \\ 7, 6 \end{aligned}$$

9) Mapping Theory to Your Python File

In 03_bresenham_circle.py:

1. circle_symmetric_points(...)
 - computes and deduplicates the 8 mirrored points.
2. bresenham_circle(...)
 - generates circle points from decision parameter logic.
3. bresenham_circle_trace(...)
 - returns each step with p and decision for debugging/teaching.



This makes your file excellent for both drawing and understanding the algorithm.

10) Quick Pseudocode

```
function bresenham_circle(xc, yc, r):
    if r == 0: return [(xc, yc)]

    x = 0
    y = r
    p = 3 - 2*r
    points = symmetric_points(xc, yc, x, y)

    while x <= y:
        if p < 0:
            p = p + 4*x + 6
        else:
            p = p + 4*(x - y) + 10
            y = y - 1

        x = x + 1
        points += symmetric_points(xc, yc, x, y)

    return points
```

11) Practical Use Cases

1. Basic CAD and plotting tools
 2. Game mini-maps and radar circles
 3. UI rings, knobs, and circular indicators
 4. Teaching rasterization fundamentals
-

12) Short Revision Summary

1. Circle algorithm starts at (0, r).
2. p decides if y stays or decreases.

3. Update with integer formulas only.
4. Plot one octant, mirror to 8 octants.
5. Stop when x crosses y .

This is the classic and efficient way to rasterize circles.